Instructor: Dov Kruger

2026 Fall

Contents

1 Instructor: Dov Kruger

2 Outline

3 What is CUDA?

4 CUDA Programming Model

5 CUDA Terminology

6 2D Kernel Indexing

7 Thread Indexing and Shared Memory

8 CUDA Model Summary

9 GPU Memory Hierarchy Overview

10 Latency and Bandwidth — Typical Ranges

11 Representative Device Bandwidths

12 Why On-Chip Memory Matters

13 GDDR vs HBM

14 GDDR and HBM Evolution

15 Energy and Voltage Trends

16 Thread and Block Indexing

17 Memory Hierarchy — Scope and Latency

18 Quantitative Memory Levels — RTX Class GPUs

19 CUDA Memory Types Explained

20 RTX 2060 — TU106 2019

21 RTX 3060 — GA106 2021

22 RTX 4060 — AD107 2023

23 RTX 5060 — Blackwell projected

24 Memory Coalescing and Shared Memory

25 Atomics and Reductions

26 Streams and Concurrency

27 Unified / Managed Memory

28 Dynamic Parallelism

29 Roofline Model for GPUs

30 Occupancy and Latency Hiding

31 Instruction vs Memory Throughput

32 Profiling — Nsight

33 Instruction-Level Parallelism

34 Parallel Reductions and Scan

35 Memory-Bound vs Compute-Bound

36 FFT on GPUs

37 Matrix Multiplication

38 Sorting on GPUs

39 RNG on GPU

40 Memory Footprint and Occupancy

41 Merge-in-Registers Idea

42 Practical Exercises

Outline

- Part I — Foundation and architecture
- Part II — CUDA programming model and fundamentals
- Part III — Memory hierarchy and device comparison
- Part IV — Exotic techniques, roofline, and application patterns

What is CUDA?

- CUDA = Compute Unified Device Architecture
- NVIDIA parallel computing platform and API
- General-purpose programs on the GPU (not only graphics)
- Thousands of lightweight threads; high throughput for FFT, matmul, simulation
- Proprietary to NVIDIA; AMD analogue is HIP
- No single portable programming model for all GPUs

CUDA Programming Model

- Code runs on host (CPU) and device (GPU)
- GPU executes kernels in parallel
- Launch: grid of blocks, each block has threads
- Same kernel code, different data per thread
- Memory: registers, shared memory, global memory



CUDA Terminology

- Thread: smallest unit; private registers; threadIdx
- Block: thread group; shared memory; blockIdx
- Grid: all blocks launched by one kernel
- Warp: 32 threads executed in lock-step on an SM
- SM: streaming multiprocessor — warps, registers, shared SRAM
- Kernel: __global__ function executed on device
- Host / device: CPU / GPU
- Occupancy: active warps vs hardware maximum on an SM
- Warp divergence: threads in a warp take different control paths

2D Kernel Indexing

```text
// 2D grid mapping: one thread per pixel (teaching example)
__global__ void kernel(float* img, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int idx = y * width + x;
    if (idx < width * height)
        img[idx] = idx;
}
```

Thread Indexing and Shared Memory

- Global 1D index: threadIdx.x + blockIdx.x * blockDim.x
- 2D: separate x/y from blockIdx, threadIdx, blockDim
- Shared memory: fast block-local scratchpad
- Global memory: all threads; high latency — coalesce accesses

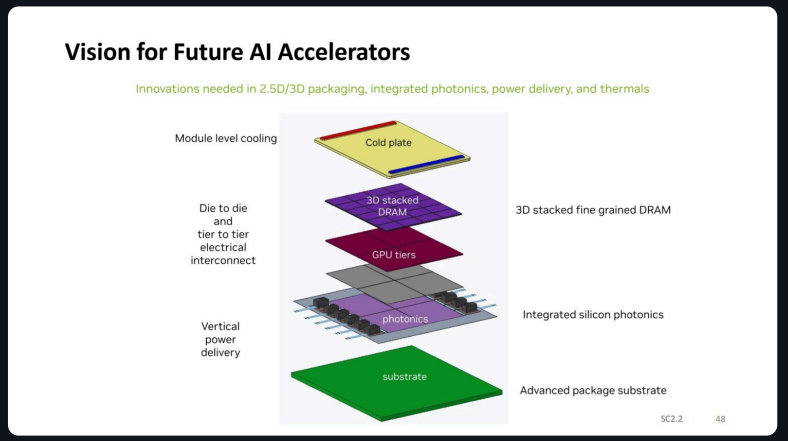
```text
// Shared-memory staging (fragment — not a complete kernel)
extern __shared__ float sharedData[];
// tid = global linear index
int tid = threadIdx.x + blockIdx.x * blockDim.x;
sharedData[threadIdx.x] = input[tid];
__syncthreads();
```

CUDA Model Summary

- Threads grouped in blocks, blocks in grids
- Warps execute in lock-step (32-wide)
- Minimize global traffic; prefer registers and shared memory
- Deterministic SIMT parallelism at scale

GPU Memory Hierarchy Overview

- Registers: per-thread, fastest, tiny
- Shared / L1: on-SM SRAM, block scope
- L2: device-wide cache between SMs and DRAM
- Constant / texture: read-only, specialized caching
- Global: off-chip GDDR or HBM — capacity and bandwidth
- Memory coalescing is critical for global throughput



Latency and Bandwidth — Typical Ranges

- Registers: about 1–2 cycles
- L1 / shared: about 20–30 cycles
- L2: about 150–300 cycles
- Global: about 200–1000 cycles (roughly 100–600 ns)
- Constant: fast broadcast when all threads read same address; else global-like

Representative Device Bandwidths

- GTX 1050 (GDDR5): 100–120 GB/s
- RTX 2060 (GDDR6): 300–350 GB/s
- RTX 3060 (GDDR6): 350–400 GB/s
- Tesla T4 (GDDR6): 300–400 GB/s
- RTX 4090 (GDDR6X): 900–1100 GB/s

Why On-Chip Memory Matters

- Global latency dominates when data is not reused
- Register and shared reuse cuts off-chip bytes moved
- Large sequential transfers amortize latency toward peak bandwidth
- Lower-bandwidth GPUs benefit more from in-SM tuning

GDDR vs HBM

- GDDR: high per-pin data rate; lower cost; wide PCB routing
- HBM: stacked dies, very wide interface, lower energy per bit
- GDDR: consumer GPUs; HBM: HPC / AI bandwidth and efficiency
- Trend: GDDR raises pin rate; HBM widens bus and cuts pJ/bit

GDDR and HBM Evolution

- GDDR: GDDR5 → GDDR5X → GDDR6 → GDDR6X (and GDDR7 emerging)
- HBM: HBM1 → HBM2 → HBM2e → HBM3 → HBM3E

Energy and Voltage Trends

- GDDR: about 1.1–1.35 V; several pJ/bit
- HBM: about 1.1–1.2 V or lower; fewer pJ/bit — better perf/W
- HBM: short interconnects, TSVs, interposer reduce capacitance

Thread and Block Indexing

- 1D: idx = threadIdx.x + blockIdx.x * blockDim.x
- 2D: x/y from blockIdx.*, threadIdx.*, blockDim.*
- 3D: same pattern for volumetric data
- Maps threads to array, image, or volume elements

Memory Hierarchy — Scope and Latency

- Fastest → slowest: registers → shared/L1 → L2 → global DRAM
- Registers: few cycles; global: hundreds of cycles
- Constant and texture caches: read-only, specialized access patterns

Quantitative Memory Levels — RTX Class GPUs

Representative ranges (2060–5060 families); not one SKU.

Memory	Latency (cycles)	Scope	Capacity (typical)
Registers	1–2	Thread	64–256 KB / SM
Shared / L1	20–30	Block / SM	64–192 KB / SM
L2	150–300	Device	2–96 MB
Global GDDR	400–1000	Device	6–24 GB
Constant / texture	10–100 cached	Device	64 KB const.

CUDA Memory Types Explained

- Registers: private per thread
- Shared memory: programmer-managed on-chip (per block)
- L1: hardware cache; fused with shared on recent GPUs
- L2: unified cache shared by all SMs
- Global: main device DRAM (GDDR/HBM)
- Constant: read-only broadcast cache (~64 KB)
- Texture: 2D/3D locality optimized, read-only
- Static: driver-reserved L2 metadata (conceptual budget)

RTX 2060 — TU106 2019

Attribute	Value	Notes
SMs	30	Turing
CUDA cores	1920	64 / SM
Clock	1365–1680 MHz	
Registers / SM	64K × 32-bit	256 KB / SM
L1 / shared	64 KB configurable	Combined pool
L2	3 MB	Device-wide
Global	6 GB GDDR6	

RTX 3060 — GA106 2021

Attribute	Value	Notes
SMs	28	Ampere
CUDA cores	3584	128 / SM
Clock	1320–1777 MHz	
L1 / shared	128 KB / SM	Configurable split
L2	3 MB	
Global	12 GB GDDR6	

RTX 4060 — AD107 2023

Attribute	Value	Notes
SMs	24	Ada Lovelace
CUDA cores	3072	128 / SM
Clock	1830–2460 MHz	Boost dependent
L1 / shared	128 KB / SM	
L2	24 MB	Much larger than 3060
Global	8 GB GDDR6	

RTX 5060 — Blackwell projected

Attribute	Estimate	Notes
SMs	32–36	Blackwell
CUDA cores	4096–4608	128 / SM
Clock	2000–2600 MHz	
L1 / shared	128–192 KB / SM	
L2	32–48 MB	
Global	12 GB GDDR7	

Memory Coalescing and Shared Memory

- Coalesced global accesses: one cache line served to a warp efficiently
- Shared memory: fast intra-block communication
- Avoid shared bank conflicts (stride patterns)
- __syncthreads() before consuming data other threads wrote

Atomics and Reductions

- Atomics: thread-safe updates to global or shared locations
- Reductions: sum / min / max across threads — atomics or tree reduce
- Minimize atomic contention (privatize, then merge)

Streams and Concurrency

- CUDA streams: ordered queue of ops on the device
- Overlap kernels with H2D/D2H copies
- Multiple streams hide latency for memory-bound work

Unified / Managed Memory

- Single pointer visible on host and device (cudaMallocManaged)
- Simpler code; migration on demand
- Performance depends on access pattern and page faults

Dynamic Parallelism

- Device code can launch kernels (__device__ launches)
- Useful for adaptive / recursive algorithms
- Can reduce occupancy — use sparingly

Roofline Model for GPUs

- Peak bounded by compute or memory bandwidth
- Operational intensity: FLOPs per byte moved from DRAM
- Plot achievable GFLOP/s vs intensity; knee shows bound type

Occupancy and Latency Hiding

- More resident warps help hide memory latency
- Limited by registers / thread and shared memory / block
- Tune block size and resource use for throughput

Instruction vs Memory Throughput

- SMs can issue many instructions per cycle when work is ready
- Memory-bound kernels plateau at DRAM bandwidth
- Coalescing and sequential access raise effective GB/s

Profiling — Nsight

- Kernel time, achieved bandwidth, occupancy
- Stalls, warp divergence, register spills to local memory
- Use profiles to choose next optimization (not guesswork)

Instruction-Level Parallelism

- Independent instructions in one thread can overlap
- Hides arithmetic and short-memory latency inside a thread
- Most important when compute-bound

Parallel Reductions and Scan

- Reduction: tree in shared memory; warp shuffle on modern CUDA
- Scan / prefix sum: stream compaction, histograms, sort building block
- Hierarchical: block scan, then inter-block fixup

Memory-Bound vs Compute-Bound

- Memory-bound: performance follows global bandwidth
- Compute-bound: follows peak FLOP/s (CUDA cores / tensor cores)
- Roofline identifies which applies

FFT on GPUs

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}$$

- Naive DFT: one output per thread — memory-bound for large N
- Optimized: shared tiles, coalescing, radix / warp primitives

Matrix Multiplication

- Naive: each thread one output — poor reuse
- Tiled: shared-memory blocks of A and B
- Tensor cores (Ampere+): mixed-precision GEMM acceleration

Sorting on GPUs

- Radix sort, bitonic networks — common GPU sorts
- Merge sort: hard to parallelize; memory traffic heavy
- Small sorting networks: register-resident, very fast

RNG on GPU

- Per-thread deterministic RNG for demos (Philox, counter-based)
- Reproducible FFT / spectral experiments
- Avoid correlated streams across threads

Memory Footprint and Occupancy

- Registers and shared memory per block cap occupancy
- Trade fewer registers vs more concurrent blocks
- Double-buffer shared tiles to overlap load and compute

Merge-in-Registers Idea

- Sort small register-resident runs (e.g. 8 values)
- Merge local buffers; refill with sequential global reads
- Goal: sequential DRAM traffic, fewer bytes off-chip

Practical Exercises

- Complete scaffold in code/cuda/sum.cu (cudaMalloc, launch config)
- Counter-based RNG + small FFT
- Tiled matmul vs naive — compare Nsight bandwidth
- Register vs shared merge micro-kernels

Summary

- CUDA: host launches kernels on a grid of thread blocks
- Warps of 32 threads — divergence and coalescing matter
- Memory hierarchy spans registers to HBM/GDDR
- Profile, roofline, then optimize occupancy and memory traffic